IFCD107 · ESPECIALISTA EN INTELIGENCIA ARTIFICIAL

Predecir un precio con una recta

Regresión lineal: coeficientes en euros, residuos e interacciones

Sprint 7 · S07_01 · 40 min en clase (ejercicios 1, 2 y 3)

LA IDEA · SECCIÓN 1

Una suma de efectos que se lee en euros

Ejemplo con 8 reservas (2 por casilla; aquí, su precio medio). **Regresión** = predecir un número, el precio de la noche.

tipo_habitacion	tipo_suite	alta	precio medio
estandar	0	0	70 €
estandar	0	1	84 €
suite	1	0	150 €
suite	1	1	186 €

LO QUE APRENDE EL MODELO

```
precio = 64.5
       + 91.0 * tipo_suite
       + 25.0 * alta
```

Coeficiente

Euros que cambia el precio al subir esa variable en 1, **con las demás iguales**.

Dummy: columna 0/1

Una categoría pasa a 0/1. **estandar** queda como **referencia** (todo a 0).

Se puede explicar

Por eso es el primer modelo que se prueba: cada número tiene sentido.

MAE, RMSE y R^2: tres formas de medir

real	predicho	error (valor absoluto)
80 €	75 €	5 €
95 €	100 €	5 €
120 €	110 €	10 €
70 €	72 €	2 €
150 €	160 €	10 €

6,4 €

MAE = media de los errores de estas 5 reservas: (5+5+10+2+10) / 5. Lo que te equivocas, de media, **en euros**.

7,1 €

RMSE: como el MAE, pero los errores grandes pesan más (elevar al cuadrado).

0,94

R^2: 1 = perfecto; 0 = no mejor que predecir la media; negativo = peor.

AJUSTAR · EJERCICIO 1

Entrenar, predecir y comparar con el baseline

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=SEMILLA)
modelo_lin = LinearRegression().fit(X_train, y_train)   # ajustar con train
pred = modelo_lin.predict(X_test)                       # predecir en test
r2_test = r2_score(y_test, pred)
mae_test = mean_absolute_error(y_test, pred)
```

0,915
R² en test

7,24 €
MAE en test

9,70 €
RMSE en test

Baseline = predecir siempre la media: MAE 25,56 € y R² ≈ 0. Nuestro modelo baja el error a 7,24 €.

INTERPRETAR · EJERCICIO 2

Cada coeficiente es una diferencia en euros

```
coefs = pd.Series(modelo_lin.coef_, index=X.columns)
extra_suite = coefs["tipo_habitacion_suite"]
efecto_temporada_alta = coefs["temporada_alta"]
```

Con las 8 reservas del ejemplo	Valor	Cómo se lee
tipo_suite	91.0	Una suite cuesta 91 € más que una estándar
alta	25.0	Reservar en temporada alta añade 25 €
intercept_	64.5	Precio base: estándar en temporada baja

En el cuaderno (6 000 reservas)

Suite **+101,2 €** · alta **+22,4 €** · base **67,3 €**

Ojo con la lectura

Una dummy **no es el precio**: es la diferencia con la referencia.

RESIDUOS · SECCIÓN 3

¿Se equivoca de forma «sana»?

Residuo = real − predicho. Si cuesta 82 € y el modelo dice 89,5 €, el residuo es −7,5 €: se pasó.

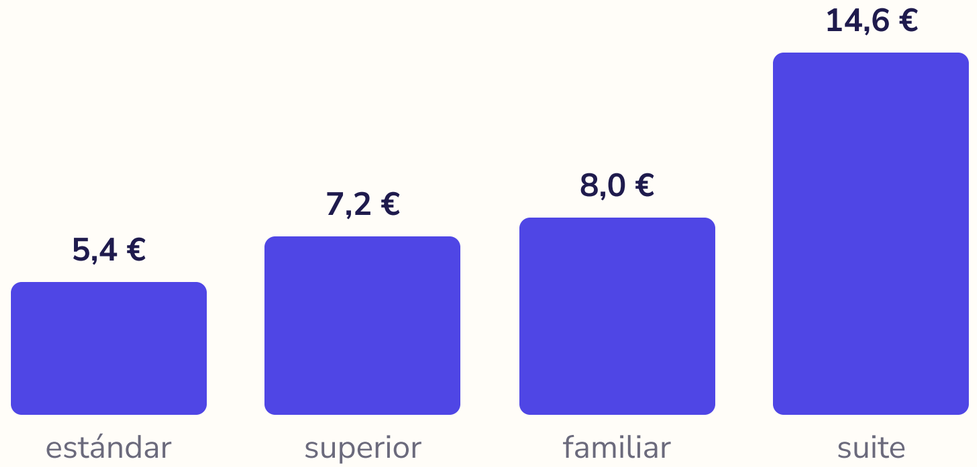
Modelo bien planteado

Residuos **sin patrón**, repartidos alrededor de 0.

Un embudo

Los errores **crecen con el precio**: al modelo le falta algo.

Error medio (MAE) por tipo de habitación, en test (mismo modelo):



El modelo suma euros fijos; los precios cambian en proporción (la temporada alta los multiplica por 1,25).

INTERACCIONES · SECCIÓN 4

El efecto de la temporada depende de la habitación

Precio medio	Temporada baja	Temporada alta	La temporada añade
estándar	70 €	84 €	+14 €
suite	150 €	186 €	+36 €

Sin interacción

El modelo pone el mismo **+25 €** a las dos. Suite en alta: predice 180,5 €; real, 186 €.

Con interacción

Añade una columna **tipo_suite · alta**, con coeficiente 22 (36 menos 14). Suite en alta: **186 €**.

Interacción = producto de dos variables: deja que el efecto de una dependa de la otra.

INTERACCIONES · EJERCICIO 3

PolynomialFeatures crea los productos

```
con_interacciones = make_pipeline(
    PolynomialFeatures(degree=2, interaction_only=True, include_bias=False),
    LinearRegression())
con_interacciones.fit(X_train, y_train)
mae_inter_test = mean_absolute_error(y_test, con_interacciones.predict(X_test))
mejora_mae = mae_test - mae_inter_test   # positivo = mejor
```

es_suite	alta	→	es_suite	alta	producto
1	1		1	1	1
1	0		1	0	0
0	1		0	1	0
0	0		0	0	0

Resultado en el cuaderno

R^2 (CV): **0,924 → 0,933**

MAE en test: **7,24 € → 6,76 €**

POLINOMIOS · EJERCICIO 4 (CASA)

Más complejidad no siempre gana

POTENCIAS DE X

```
PolynomialFeatures(3)
x = 2  →  1, 2, 4, 8
x = 3  →  1, 3, 9, 27
# columnas: 1, x, x², x³
```

Habitaciones estándar (CV)	R^2
Mes crudo, grado 2	0,09
Mes crudo, grado 8	0,55
Mes crudo, grado 12	0,60
temporada_alta (1 columna)	**0,65**

Pensar bien la variable (feature engineering) rinde más que añadir complejidad: una columna bate al grado 12.

Con el mes es fácil que el polinomio de grado alto **sobreajuste** y, además, no se puede explicar al cliente.

MINI-RETO (CASA)

La tarifa de referencia, explicada al cliente

precio_recomendado(tipo, alta) construye **una fila con las mismas columnas que X** (mismo orden) y se la pasa a **modelo_lin.predict**.

tipo, alta	tipo_familiar	tipo_suite	tipo_superior	alta
estandar, 0	0	0	0	0
suite, 1	0	1	0	1
superior, 1	0	0	1	1
familiar, 0	1	0	0	0

Qué comprobar

Una estándar en temporada baja ronda **70 €**; una suite en temporada alta pasa de **180 €**.

La estándar es la referencia

No tiene columna que activar: su fila queda con los tres tipos a 0.

HOY HAS APRENDIDO

Explicar el precio con números

Interpretar

Coeficientes en euros y la categoría de referencia

Medir

MAE, RMSE y R^2 frente al baseline, y residuos sin patrón

Mejorar

Interacciones y variables bien pensadas, antes que más complejidad

Siguiente: S07_02 · regularización con Ridge y Lasso